MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY, ALLAHABAD
DEPARTMENT OF MECHANICAL ENGINEERING

IoT-BASED AUTOMATED SMART IRRIGATION SYSTEM

Using NodeMCU ESP8266 and Self-Hosted Blynk Legacy Infrastructure

PROJECT RESEARCH PAPER

IoT Project Instructor: ***Mandeep Dhanda***
MNNIT Allahabad

Submitted by

Anuj Parihar (20247015)
Sudarshan Jha (20247051)
Mayank Gupta (20247027)
Aman Kharwar (20247010)

ABSTRACT

Irrigation is most useful when the water supplied to a plant reflects the moisture condition around its roots. Fixed schedules can continue watering when soil is already damp, while manual inspection may miss a drying root zone. This paper presents the design and current implementation status of a low-cost Internet of Things (IoT) irrigation prototype built around a NodeMCU ESP8266, an analog soil-moisture sensor, an active-LOW relay module, and a separately powered DC pump. The sensor is connected to analog input A0. The current firmware compares the raw ADC reading with a working threshold of 400: a value above the threshold is treated as a dry-soil condition and commands the relay ON by driving D3 LOW; otherwise, D3 is driven HIGH and the pump remains OFF. The threshold is provisional and requires calibration in the intended soil.

A second contribution of the project is the recovery of a discontinued Blynk Legacy workflow through a community-maintained Android application and a self-hosted Java server fork. The server is provisioned on a local network, with port 9443 used for secure administration and port 8080 available for hardware TCP communication. This approach gives the team control over the local test environment and avoids a recurring cloud subscription for the prototype. It should not be confused with a currently connected remote-control system: the local server and mobile setup have been prepared, but the Blynk.config() and Blynk.run() statements in the current firmware remain commented pending the final server IP and device integration.

The paper describes the motivation, system architecture, component interfaces, local-server deployment, firmware decision logic, planned sensor calibration, validation method, safety requirements, limitations, and possible expansion to multi-node agriculture using ESP-NOW or LoRaWAN. The two supplied project images are included as illustrative references rather than proof of completed hardware testing. No measured water-saving percentage or calibrated volumetric water content is claimed. The next engineering step is to complete the physical wiring, verify the pump nameplate and relay ratings, validate dry and wet readings in the target soil, and test the complete control loop under safe conditions.

Keywords

Internet of Things (IoT), smart irrigation, NodeMCU ESP8266, soil-moisture sensing, Blynk Legacy, self-hosted server, local network, relay control, automatic watering, manual override, telemetry, sensor calibration, ESP-NOW, LoRaWAN, power optimization.

Paper organization

The remainder of the paper moves from the agricultural problem and related work to the system architecture, hardware and server infrastructure, mobile interface, firmware logic, calibration plan, implementation status, and future development. The current control logic is explained within the main paper so that the design decisions and their limitations can be reviewed in context.

I. INTRODUCTION AND PROJECT OBJECTIVES

Reliable irrigation is not simply a matter of supplying a fixed quantity of water at a fixed time. The useful amount depends on the plant, root-zone condition, soil texture, container size, drainage, and local weather. A timer-based system can therefore run when the soil is already moist, while manual watering may be delayed when a person is unavailable.

Both situations can reduce the quality of plant care and waste water.

A small IoT node can improve this process by combining sensing, local decision-making, and an actuator. In the present project, a soil-moisture module produces an analog signal. The NodeMCU ESP8266 reads that signal through A0, compares it with a configured threshold, and controls a relay connected to the pump circuit. Wi-Fi connectivity and a Blynk-based mobile interface are intended to provide a convenient way to view or command the device when the local server integration is completed.

A. Project motivation

The project was selected because water management is a practical engineering problem that can be explored with affordable components. It also gives the team experience with embedded programming, analog sensing, relay switching, local networking, application interfaces, and system documentation. A prototype at plant scale does not solve agricultural water management by itself, but it provides a controlled way to demonstrate the principle of feedback-based irrigation.

The national context makes efficient irrigation relevant. In the 2025 Dynamic Ground Water Resources Assessment, the Ministry of Jal Shakti reported that 215 BCM, or about 87% of India's annual groundwater extraction, was used for irrigation [8]. This figure describes national groundwater use; it is not a measurement of water use by this prototype. It helps explain why even small improvements in irrigation practice deserve attention.

B. Problem statement

The engineering problem addressed in this work is: how can a low-cost controller use a soil-moisture signal to decide when to start or stop a pump, while keeping the actuator electrically separate from the microcontroller and providing a path to local IoT monitoring?

C. Objectives

- Acquire soil-moisture readings through the NodeMCU analog input A0.
- Use a clear threshold-based decision rule to identify a dry-soil condition.
- Drive an active-LOW relay through digital pin D3 while keeping pump current outside the GPIO circuit.
- Prepare a self-hosted Blynk Legacy server and a compatible mobile interface on the local network.
- Document the current status honestly, including functions that remain untested or pending.
- Define a calibration and validation process before making claims about water savings or measurement accuracy.

D. Scope of the present prototype

The present scope is a single-node plant-watering prototype. It does not yet include multi-zone field deployment, automatic weather compensation, water-flow measurement, a reservoir-level sensor, or a validated agronomic moisture threshold. These are treated as possible extensions rather than completed features. The design intentionally keeps the first control loop understandable so that its behavior can be tested and improved step by step.

E. Why use local decision-making?

The control decision is kept at the edge, on the ESP8266, rather than being sent to a cloud service for every sensor sample. This keeps the basic irrigation response simple and reduces dependence on Internet availability. The network layer is useful for visibility and user interaction, but it should not become a single point of failure for the local moisture rule. If the local device is running and its sensor and relay circuit are correctly powered, the automatic decision can continue even when the mobile application is not open.

F. Expected engineering output

The expected output is not only a pump switching demonstration. The team intends to document the signal path, the software decision, the electrical boundary between controller and actuator, and the steps needed to reproduce the setup. This makes it possible for another student to understand why the pump is commanded, where the threshold is stored, which parts of the system are network-dependent, and which tests must be completed before unattended operation is considered.

For evaluation, the distinction between a prepared feature and a verified feature is important. A server console can be accessible while the microcontroller is not yet connected. A dashboard widget can be configured while its firmware callback is still inactive. Likewise, a firmware statement can command a relay output without proving that the final pump circuit is wired safely. The report uses these distinctions throughout.

II. RELATED WORK AND DESIGN CONTEXT

Soil-water sensing is a well-established method for supporting irrigation decisions. In contrast to timer-only operation, a sensor-based controller uses information from the growing medium to decide whether irrigation is needed. A network connection can add remote visibility and user commands, but the local controller should still make the basic safety and irrigation decision without depending on a distant server for every sample.

A. Evidence from sensor-based irrigation

Datta and Taghvaeian reviewed soil-water sensors used for irrigation scheduling and found that sensor-based scheduling generally used less water than the comparison methods in the studies they examined. Reported average savings ranged from about 1% against canopy-temperature methods to 38% against traditional practices [1]. The spread is important: the result changes with crop, climate, soil, baseline practice, sensor placement, and calibration. It would therefore be misleading to present 30-50% as a guaranteed saving for the current prototype.

B. Calibration and uncertainty

Kanso et al. studied EC5 and 5TE capacitance sensors in urban soils and showed that sensor readings depend on calibration and soil conditions [2]. Their instruments are not the same as the low-cost module used in this project, so their

accuracy figures cannot be transferred to this design. Their work supports a more general point: a displayed moisture percentage must be tied to a stated calibration method and a particular substrate.

C. Related IoT implementations

Abd Halim et al. described a NodeMCU ESP8266 testbed using Blynk and Thinger.io for chilli-plant monitoring and control [3]. Saputri et al. presented a NodeMCU-based arrangement that used Blynk for monitoring and control and ThingSpeak for historical data handling [4]. Pereira et al. reported an ESP32-based drip-irrigation system combining Blynk with additional environmental and flow sensing [5]. Together, these studies show common design choices: an edge controller, moisture-driven actions, a user-facing dashboard, and the value of collecting more than a single sensor reading.

D. Research gap and contribution

The current project focuses on a low-component-count architecture that can be explained and demonstrated with a single ESP8266 analog channel. Its distinctive engineering task is not a new sensing principle; it is the combination of a simple moisture-triggered control loop with the recovery of a discontinued Blynk Legacy workflow through a self-hosted server. The project also makes implementation status explicit. Server provisioning and mobile-interface preparation are documented, while final hardware integration, device-to-server firmware connection, and calibrated irrigation trials remain pending.

E. Design implications

The literature informs three decisions in this project. First, the moisture sensor must be calibrated in the actual soil rather than treated as a universal instrument. Second, the system should separate local automatic control from network communication so that a temporary network problem does not prevent the basic sensing loop from running. Third, future evaluation should record water volume and pump run-time instead of relying only on a dashboard percentage.

F. Design lessons from the literature

The studies point to complementary improvements. Sensor scheduling addresses when to irrigate; calibration addresses whether a reading can be trusted; flow measurement addresses how much water is actually delivered; and a dashboard improves visibility for the operator. A compact educational prototype cannot implement every layer at once. It should first produce repeatable raw sensor readings and safe relay behavior, then add network reporting and richer analytics after the local circuit has been validated.

G. Relevance of the Legacy platform problem

The Blynk Legacy recovery task is a software-infrastructure issue rather than an agronomic contribution. It is relevant because many educational examples were written for a particular application and server generation. When those services are withdrawn, a previously familiar sketch may no longer be reproducible by a new user. A local server can preserve a learning environment for a controlled prototype, but its unsupported status also introduces maintenance and security limitations that must be acknowledged.

The literature values cited above describe the referenced studies and are not measured results from the MNNIT prototype.

III. SYSTEM ARCHITECTURE AND DATA FLOW

The system is organized around four functions: sensing, edge processing, actuation, and local IoT communication. The sensor observes the soil around the probe. The NodeMCU converts the analog output into a digital reading and applies the control rule. A relay switches the pump supply circuit. A local Blynk Legacy server and compatible mobile application form the planned communication layer for status display and user commands.



Fig. 1. Illustrative plant-scale smart-irrigation arrangement showing a soil probe, controller, relay, pump, and mobile interface. This image is a visual reference, not photographic evidence of the team's completed wiring.

A. Sensing layer

The soil-moisture module is connected to the NodeMCU analog input A0. The module produces a voltage that changes with the electrical properties of the surrounding soil. For the sensor behavior assumed in the current firmware, a higher ADC count indicates drier conditions and a lower count indicates wetter conditions. The relationship must be confirmed with the exact sensor module and soil used in the final test.

B. Edge-processing layer

The ESP8266 reads A0 using analogRead(). The returned integer is compared with the current working threshold of 400. If the reading is greater than 400, the controller treats the soil as dry and commands the relay ON. Otherwise, it commands the relay OFF. This local decision does not require a cloud response, so the core automatic loop can operate independently of the dashboard once the firmware is running.

C. Actuation layer

Digital pin D3 controls the input of an active-LOW relay module. Driving D3 LOW energizes the relay; driving it HIGH releases the relay. The relay contacts, rather than the

NodeMCU pin, switch the pump's separate power circuit. The intended idle condition is pump OFF.

D. Communication layer

The local server is intended to sit on the same private network as the device and mobile client. The server provides a route for telemetry and commands, but this layer is not yet active in the current sketch because the local IP configuration and Blynk.run() call remain commented. The paper therefore treats app-to-device operation as planned integration, not a completed live demonstration.

E. End-to-end flow

Soil condition → sensor output → A0 sampling → threshold comparison → D3 relay signal → pump switching → change in soil moisture. The last step closes the feedback loop: after irrigation, a later sensor reading should show whether the soil condition has changed enough for the controller to stop the pump.

F. Automatic operation and user-requested operation

The automatic path is designed to be local: the sensor reading is evaluated by the microcontroller and the relay output follows the threshold rule. The mobile interface is intended to add an operator-requested command. These are different control paths and should be kept distinct in the firmware. When manual override is implemented, the code should define whether manual commands are permitted in every mode, whether automatic mode has priority, and how the device returns to a safe state after a disconnection or restart.

G. Fault and recovery path

A practical system must also decide what happens when the sensor is disconnected, the ADC value is outside the expected range, the pump supply is absent, or the server cannot be reached. A sensible first revision is to leave the pump OFF during boot and treat clearly invalid sensor values as a fault rather than as a request for indefinite watering. A maximum run-time and an alarm condition should be added before the device is left unattended.

IV. HARDWARE COMPONENTS AND INTERFACING

The prototype uses a small set of commonly available modules. The precise wiring must be checked against the board revision, relay-module markings, sensor supply requirements, and the pump nameplate. Board clones can differ in their analog-input range and regulator arrangement, so a pin label alone is not a sufficient safety check.

Component roles and interfaces

NodeMCU ESP8266: main controller, Wi-Fi interface, analog input A0, and digital relay-control output D3. Confirm the actual A0 voltage limit for the exact board revision.

Soil-moisture module: VCC and GND provide power and reference; analog output AO connects to A0. The digital comparator output, if present, is not used by the current firmware. Do not exceed the board's analog-input voltage range.

1-channel relay: input IN connects to D3; the relay contacts switch the pump supply. Confirm active-LOW behavior, coil supply, isolation, and contact rating from the actual module markings.

DC water pump: connected through relay COM/NO contacts and powered from its own correctly rated supply. Verify rated voltage, current, inrush, and polarity from the pump nameplate.

USB/controller supply: powers the NodeMCU during programming and operation. The pump must not be powered from the NodeMCU 3.3 V rail.

A. Sensor interface

The sensor's analog output is used because the program needs a changing numerical value rather than only a preset wet/dry comparator output. The digital output pin, if present on the sensor module, is not used by the current firmware. The analog signal is read at A0. The controller and sensor must share the required low-voltage reference ground, subject to the module's wiring instructions.

B. Relay and pump interface

The relay acts as an electrically controlled switch. For a pump that should remain OFF in the normal state and turn ON when the relay is energized, the intended contact arrangement is generally COM and NO, provided this matches the relay module and load design. The NodeMCU GPIO drives only the relay input; it must never carry the pump current. The pump must use a separate supply selected from its verified nameplate.

C. Electrical safety

- Verify whether the pump is DC or AC and confirm its rated voltage, current, and starting/inrush demand.
- Select relay contacts and wiring for the actual load; a generic resistive-load rating may not be suitable for a motor.
- Keep pump power wiring physically separate from sensor, USB, and microcontroller signal wiring.
- Mount the relay and any pump power connections in an insulated enclosure with strain relief and suitable protection.
- Disconnect all supplies before changing wiring. Any mains-voltage installation must be enclosed and checked by a qualified person.

D. Pin and logic summary

- Moisture sensor: A0 (analog input).
- Relay control: D3 (digital output).
- Relay logic: active-LOW; LOW = ON and HIGH = OFF.
- Working threshold: raw ADC greater than 400 is treated as dry.

E. Board-level precautions

On common NodeMCU development boards, the A0 pin is routed through a board-level voltage divider, but the permitted input range can differ between board revisions and clones. The exact board marking or schematic should be checked before connecting the sensor output. The ESP8266 chip ADC itself has a lower input range than some development-board A0

pins, so the board's divider must not be assumed without verification.

The D3 label on a standard NodeMCU maps to GPIO0, which is also one of the ESP8266 boot-configuration pins. This makes relay-module behavior during power-up important. If the relay input or external circuit pulls the pin to the wrong level while the controller resets, the board may fail to start in its normal mode. The team should verify the relay input state during boot and consider moving the relay signal to a more suitable GPIO if the actual module causes a boot problem.

F. Power separation

The NodeMCU is a low-voltage logic device and is not designed to source motor current. The pump must be powered from an independent supply that matches its rated voltage and current. The relay contacts provide the switching boundary between the low-voltage control signal and the pump circuit. Separate power paths reduce the chance that motor current transients or voltage drops disturb the controller, but correct grounding, suppression, enclosure, and ratings still need to be checked for the actual hardware.

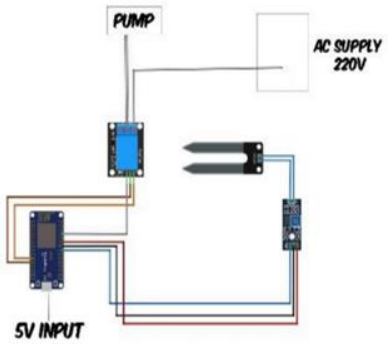


Fig. 2. Wiring reference supplied by the project team. The drawing labels a 220 V AC pump supply; it is a conceptual reference, not a finalized wiring instruction. The actual pump type and nameplate must be verified. Any mains-voltage installation must be enclosed and checked by a qualified person.

V. BLYNK LEGACY RECOVERY AND LOCAL SERVER DEPLOYMENT

A major project challenge was that the original Blynk Legacy service is no longer supported as a public cloud platform. Blynk's migration documentation records the closure of new Legacy registrations, removal of the Legacy apps from the major app stores, and shutdown of the Legacy cloud server on 31 December 2022 [6]. As a result, older tutorials and sketches that expected the public Legacy cloud could not be used unchanged for a new demonstration.

A. Why a local server was selected

The team chose to rebuild the Legacy client-server arrangement on a private network rather than move the prototype to a subscription-based successor service. A self-hosted server gives the team direct control over the test environment and keeps the project's device traffic within the local network. It also avoids a recurring cloud subscription for this prototype. These benefits do not make the old platform production-ready: the community-maintained server and application are legacy software and require careful access control and maintenance.

B. Server deployment

The server used is the community-maintained Java fork Peterkn2001/blynk-server, based on the discontinued Blynk Legacy Local Server [7]. The server is deployed on a machine reachable from the NodeMCU and the mobile client over the same subnet. The local administration console is opened through https://[LOCAL_SERVER_IP]:9443/admin. The actual IP address is deliberately not printed in this paper because it is specific to the team's network.

Port 9443: HTTPS application and web-administration access. The administration console is available at /admin.

Port 8080: unencrypted TCP hardware connection used by the Legacy device protocol.

C. User provisioning

1. Start the local Java server and confirm that its log reports a successful start.
2. Open the administration console from a device on the permitted local network.
3. Create a dedicated project user and generate the local authentication credentials required by the hardware client.
4. Keep the token and administrator password private; do not publish them in source code screenshots or public repositories.

D. Security and operational limits

The repository documentation warns that the Legacy server and applications are discontinued and unsupported [7]. Default administrator credentials must be changed, and administrative access should be restricted to trusted IP addresses. The server's built-in development certificate is not a substitute for a production security configuration. For a classroom prototype on a private subnet, the local server is a useful recovery path; for unattended agricultural deployment, a supported platform with maintained security updates would be a more appropriate choice.

E. Current integration status

The server deployment and account provisioning are reported as completed by the team. The local server IP has not yet been inserted into the current firmware: the planned Blynk.config(auth, IPAddress(...), 8080) statement remains commented. The device-side Blynk.run() call is also commented. Therefore, the local server is prepared, but live hardware-to-server telemetry and remote switching must still be tested.

F. Why the two ports matter

The administration port and the hardware port serve different functions. The browser-based administration console is used to manage users and inspect server state. The hardware port is the endpoint that the microcontroller library must use when it connects to the server. A correct server address with the wrong port is still a failed connection. The device and phone must also be able to reach the server address on the same network, and local firewall rules must permit the required traffic.

G. Local hosting is not the same as unlimited hosting

A self-hosted arrangement removes a vendor subscription from the prototype workflow, but it does not remove infrastructure constraints. Server performance depends on the host machine, network quality, storage, concurrent devices, and administrative maintenance. A local server also does not automatically provide remote access from outside the subnet. Such access would require a deliberate and secure network design rather than simply opening administration or hardware ports to the public Internet.

VI. MOBILE APPLICATION AND MONITORING INTERFACE

The user-facing interface is based on a community-maintained Blynk Legacy Android application that can work with the self-hosted Legacy server. The team prepared a Plant Watering project and configured a button widget for the planned relay-control interaction. The app provides a familiar way to present the concept to an evaluator: a user can view the interface, understand the intended control action, and see how a mobile client fits into the overall IoT architecture.

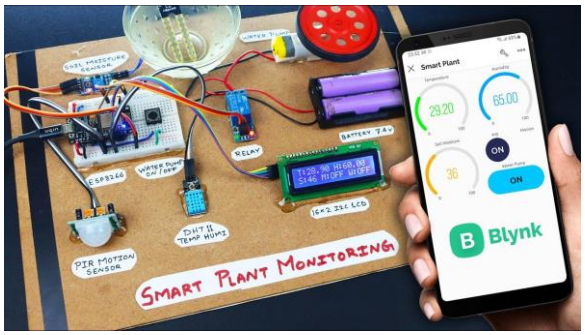


Fig. 3. Reference image of a smart-plant monitoring board and Blynk-style mobile dashboard. The pictured DHT11, PIR sensor, LCD, and battery pack are not part of the current MNNIT prototype.

A. Intended dashboard functions

The current interface preparation focuses on a project dashboard and a button widget associated with the pump-control concept. In a completed connection, the mobile application would send a command through the local server to the ESP8266, and the device would update the relay state. A moisture gauge and historical trend view could be added once the firmware publishes sensor readings to an assigned virtual pin. The exact virtual-pin mapping must be finalized together with the firmware rather than assumed from an earlier draft.

B. Manual and automatic operation

The intended user experience has two operating ideas. In automatic operation, the ESP8266 makes the pump decision from the local moisture threshold. In manual operation, a user can request pump ON or OFF from the mobile interface. The manual command should not override essential safety limits such as maximum run-time, pump dry-run protection, or a safe OFF state after a connection failure. In the current sketch, however, manual callbacks have not been added and the Blynk processing loop is commented. The app widget is therefore a prepared interface, not evidence of an active end-to-end manual override.

C. Local communication path

The intended path is Mobile App ↔ Wi-Fi / Local Network ↔ Self-Hosted Blynk Server ↔ ESP8266. The local server routes application and hardware messages; it does not replace the local sensor decision. Keeping the control rule on the NodeMCU is useful because a temporary interruption of the mobile or server connection should not be the only thing deciding whether the plant receives water.

D. Project documentation portal

The team also maintains an interactive project website at https://iotproject-nit.vercel.app. It presents the implementation story, server and application evidence, firmware section, and a status matrix distinguishing completed work from pending validation. The website is a documentation and presentation portal; it should not be described as the irrigation controller or as a substitute for the local Blynk server.

E. Telemetry design considerations

When device connectivity is enabled, the firmware and dashboard need an agreed data contract. At minimum, the moisture reading, pump command or state, and selected operating mode should be represented consistently. Each widget should have a clear unit and meaning. A value labelled "soil moisture percent" should be identified as a normalized sensor index unless it has been calibrated against a physical reference. A button should also be labelled according to whether it sends a command or simply displays the current pump state.

F. Safe interaction rules

The dashboard should avoid leaving a stale ON indication after the device disconnects. It should display connection status and the time of the last update, and it should distinguish a requested pump state from a state verified by feedback. A future version could add an alert if the pump is requested ON for too long, if the reservoir is empty, or if the moisture reading does not change after watering. Such checks would make the app more useful than a single remote button.

VII. FIRMWARE DESIGN AND CONTROL ALGORITHM

The current firmware is written in Arduino-style C++ for the ESP8266. Its active control logic is intentionally short: configure the relay output, keep the relay OFF at startup, sample the sensor, compare the reading with the threshold, and drive the relay accordingly. The ESP8266 Wi-Fi and Blynk libraries are included because the project is designed to integrate with the local server, although the connection calls are not active in the current sketch.

A. Library and pin definitions

```
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>

#define MOISTURE_PIN A0
#define RELAY_PIN D3

int moistureThreshold = 400;
```

The Wi-Fi library provides network functions for the ESP8266, while the Blynk library provides the Legacy application protocol. MOISTURE_PIN is a symbolic name for A0, where the sensor's analog output is connected. RELAY_PIN names D3, which sends the digital control signal to the relay module. The integer moistureThreshold stores the current working decision boundary.

B. Startup configuration

```
void setup() {
    pinMode(RELAY_PIN, OUTPUT);
    digitalWrite(RELAY_PIN, HIGH);

    // Blynk.config(auth,
IPAddress(LOCAL_SERVER_IP), 8080);
}
```

The setup() function runs once after power-up or reset. pinMode() configures D3 as an output. The following digitalWrite() sets it HIGH so that an active-LOW relay is released and the pump is commanded OFF at startup. The Blynk.config() line is commented because the final local server IP and device connection have not yet been inserted.

C. Repeated control loop

```
void loop() {
    // Blynk.run();
    int rawValue = analogRead(MOISTURE_PIN);

    if (rawValue > moistureThreshold) {
        digitalWrite(RELAY_PIN, LOW);
    } else {
        digitalWrite(RELAY_PIN, HIGH);
    }
}
```

loop() repeats continuously. analogRead(A0) returns a 10-bit ADC count on the board's nominal 0-1023 scale. If rawValue is greater than 400, the code interprets the soil as dry and drives D3 LOW. Because the relay is active-LOW, the relay energizes and the pump is commanded ON. Otherwise, D3 is set HIGH and the relay is OFF.

D. Version consistency and limitations

The earlier supplied report draft described a different firmware revision using D4 and 30%/60% moisture hysteresis, with Blynk Cloud connectivity. This revised paper follows the latest team brief for the current sketch: A0 sensor input, D3 relay output, active-LOW logic, and a raw ADC threshold of 400. The two revisions must not be combined. The Blynk.config() and Blynk.run() calls remain commented, and the current code does not yet implement the mobile manual-override callback. The threshold also remains provisional until soil calibration is completed.

E. Control sequence

Read A0 → compare raw ADC with 400 → if raw ADC is above 400, drive D3 LOW → otherwise drive D3 HIGH → repeat. A future refinement should add hysteresis, sensor-fault detection, and a maximum pump run-time so that small signal fluctuations do not cause rapid switching or allow a pump to run indefinitely.

F. Worked control examples

For a raw reading of 650, the condition 650 > 400 is true. The controller therefore writes LOW to D3; with an active-LOW relay, this requests the pump ON. For a raw reading of 300, the condition is false. The controller writes HIGH to D3 and requests the pump OFF. A reading exactly equal to 400 also follows the else branch because the code uses a strict greater-than comparison. These examples explain the software decision only; the actual pump response must be checked separately.

G. Improvements to the control routine

A single threshold can cause frequent relay changes when readings fluctuate around the boundary. A hysteresis design uses separate start and stop conditions: one threshold starts watering and another stops it. The gap should be selected after calibration rather than copied from an unrelated system. The code can also average several sensor samples, reject implausible values, include a maximum run-time, and publish a fault status. Network functions should be called regularly once the server connection is enabled so that communication does not block the sensing loop.

VIII. SOIL-MOISTURE CALIBRATION AND VALIDATION METHOD

An analog sensor does not directly report a universal moisture percentage. It reports a voltage that the ADC converts into a count. That count depends on the sensor design, the board's analog-input circuit, soil composition, compaction, insertion depth, temperature, supply voltage, and contact between probe and soil. For this reason, the current threshold of 400 should be treated as a working value rather than a final agronomic setting.

A. Expected sensor response

For the sensor module behavior assumed by the current sketch, wetter soil conducts more readily and produces a lower raw ADC count, while drier soil produces a higher count. The team's provisional planning references are approximately 1023 in dry air and approximately 250 under a very wet condition. These values are initial observations or targets, not validated calibration endpoints in the actual potting mix.

B. Normalized moisture index

A simple linear estimate can be written as:

Moisture index (%) = [(ADC_dry - ADC_raw) / (ADC_dry - ADC_wet)] x 100

where ADC_dry is the reading under a dry reference condition, ADC_wet is the reading under a wet reference condition, and ADC_raw is the current measurement. The result should be constrained to the 0-100 range. This is a normalized sensor index; it is not certified volumetric water content (VWC) unless it is calibrated against an appropriate soil reference, such as a gravimetric method.

C. Calibration procedure

1. Use the exact sensor, NodeMCU board, supply, and potting mix intended for the final demonstration.
2. Record repeated ADC readings in dry soil and in adequately watered soil; do not rely only on immersing the probe in water.
3. Keep probe depth, orientation, packing, and waiting time consistent between measurements.

4. At each condition, collect at least ten readings and calculate the mean and standard deviation.
5. Check that the raw reading changes consistently as water content changes. Re-select endpoints if the response is not monotonic or repeatable.
6. Choose the pump threshold from observed plant and soil behavior, then repeat the test over several watering cycles.

D. Illustrative calibration worksheet

- Dry baseline: record the stable raw ADC value to establish the dry reference.
- Slightly moist: record the response after a small, measured water addition.
- Moderately moist: record the mid-range response.
- Adequately watered: record the wet reference in the target soil, not only in free water.
- Repeat dry: repeat the baseline at the end to check drift and repeatability.

E. Validation measurements

For every trial, record the date, soil type, probe position, raw ADC count, mapped index if used, threshold decision, relay state, pump run-time, and measured water volume. A water-saving claim requires a baseline: compare the volume used by the sensor-controlled system with a defined timer-based or manual method over equivalent observation periods. The savings calculation is 100 x (V_baseline - V_sensor) / V_baseline. No saving percentage is reported until both volumes have been measured under comparable conditions.

F. Interpreting the working threshold

If the planning endpoints of approximately 1023 for dry and 250 for wet are used in the normalized equation, a raw reading of 400 maps to about 80.6% on that index. This arithmetic example is a useful warning: the raw threshold 400 should not be casually described as “40% moisture”. It is a raw count, and its percentage interpretation changes when the calibration endpoints change. The current firmware does not calculate or display this index; it compares the raw value directly.

TABLE I. Illustrative raw ADC readings and relay decisions under the current threshold rule.

Raw ADC	Interpretation / rule	D3 / pump
250	Wet-side planning reference; <= 400	HIGH / OFF
400	Threshold boundary; <= 400	HIGH / OFF
401	Just above threshold; > 400	LOW / ON
600	Dry-side example; > 400	LOW / ON
800	Drier example; > 400	LOW / ON
1023	Dry-air planning reference; > 400	LOW / ON

Note: These points illustrate the current code comparison and the project’s provisional wet/dry reference values. They are not a measured calibration dataset.

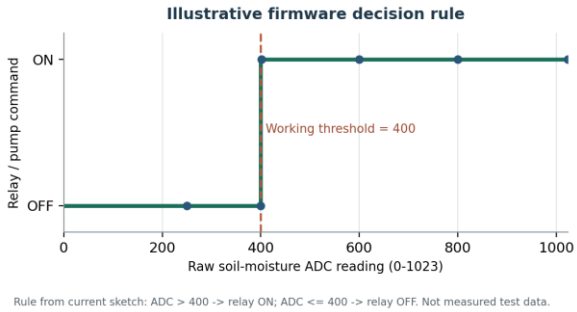


Fig. 4. Step response of the current firmware rule. The chart visualizes the programmed threshold; it does not represent measured sensor accuracy or experimental pump performance.

G. Test acceptance criteria

A basic acceptance test should confirm four things: the sensor reading changes in the expected direction as the soil is wetted; the relay input follows the code decision; the pump switches OFF during startup and on the wet-side decision; and the entire pump circuit operates without excessive heating, unstable supply, or exposed wiring. The test should be repeated across several wet/dry cycles. A relay LED alone is not an acceptable substitute for confirming actual pump operation.

IX. IMPLEMENTATION STATUS, DISCUSSION AND FUTURE DEVELOPMENT

A. Work completed by the team

The current project status is best described as a partially integrated prototype with the software and infrastructure work substantially prepared and the physical validation still in progress. The team reports that the community-maintained Java server has been deployed, the administration console has been accessed, a dedicated user has been provisioned, the community Android application has been prepared, and the mobile dashboard widget has been configured. The current firmware draft contains the automatic moisture-threshold logic and the active-LOW startup state. The technical report and interactive project website have also been prepared.

B. Work still pending

- Complete the final breadboard and low-voltage signal wiring.

- Verify the pump nameplate and select a suitable, correctly rated external supply and relay contact arrangement.
- Integrate the local server IP and authentication token into the firmware using the intended Blynk configuration.
- Enable and test Blynk.run() and the actual dashboard-to-device command path.
- Perform dry/wet calibration in the target soil and select a defensible threshold.
- Run a safe, end-to-end pump test and record water volume and run-time.

C. Interpretation of current evidence

The present evidence supports the existence of a planned local-server infrastructure, a prepared mobile client, and a written moisture-based control algorithm. It does not yet establish that the complete pump circuit has been validated under operating conditions. A relay indicator LED confirms only the relay input state; it does not prove that the pump is wired correctly or that the switched load is safe. Similarly, a sensor reading changing between dry and wet conditions does not establish calibrated VWC accuracy.

D. Scalability

The single-node design can be extended to multiple irrigation zones. Each edge node could measure the moisture of a local area and control its own low-voltage actuator, while a gateway or central server collects status information. ESP-NOW is a possible option for communication among nearby ESP-based nodes [10]. LoRaWAN is a possible option where nodes are more widely distributed and low-power, long-range telemetry is useful [11]. These are future architecture choices, not components already implemented in the current prototype. A multi-node deployment would need addressing, collision handling, gateway design, power budgeting, and field coverage tests.

E. Reliability and safety improvements

The next firmware revision should consider a maximum pump run-time, sensor-range fault detection, a safe default-OFF condition, and a cooldown interval. A reservoir-level sensor could prevent dry running, while a flow sensor could confirm that water is actually moving. For field use, a capacitive moisture sensor may reduce corrosion compared with exposed resistive probes. The local legacy server should be isolated from public Internet access and protected with non-default credentials; for long-term deployment, a maintained and supported IoT platform should be evaluated.

F. Team responsibilities

Sudarshan Jha: problem definition and infrastructure recovery strategy.

Mayank Gupta: local server deployment and user provisioning.

Aman Kharwar: hardware arrangement and firmware development.

Anuj Parihar: case study, scalability, technical documentation, reporting, and project website.

G. Potential benefits and measurement

The system can reduce unnecessary watering when its threshold is correctly selected for the plant and soil. The literature review by Datta and Taghvaeian reported savings across a broad range of comparison methods, including values up to 38% against traditional practices [1]. That result is context-dependent and should be treated only as background evidence. For this prototype, the relevant performance measures will be water volume delivered, pump run-time, sensor repeatability, and the number of correct ON/OFF decisions across controlled dry, intermediate, and wet conditions.

H. Power and deployment context

The ESP8266 requires an active supply while it samples the sensor and maintains Wi-Fi communication. Actual current draw depends on the board, radio activity, power supply, and transmission pattern, so a value should be measured on the completed build rather than copied from a generic estimate. For remote field nodes, periodic sensing and deep-sleep operation may reduce energy use. A solar panel and battery could support deployment where mains power is not convenient, but the pump energy budget must be considered separately from the microcontroller energy budget.

I. Applications

Possible applications include household plants, small greenhouses, nurseries, demonstration plots, and individual zones in a larger farm. The current single-node prototype is most suitable for a controlled plant-scale demonstration. Applying it to a field would require additional sensor locations, irrigation-zone valves or pumps, robust enclosures, weather protection, and a communication plan.

X. CONCLUSION AND REFERENCES

A. Conclusion

This project presents a practical starting point for a low-cost IoT irrigation system using NodeMCU ESP8266, analog soil-moisture sensing, an active-LOW relay, and a separately powered pump. The current control rule is easy to explain: the controller reads A0, compares the raw ADC value with a working threshold of 400, and uses D3 to command the relay. The project also addresses a separate infrastructure problem by preparing a community-maintained Blynk Legacy application and a locally hosted Java server rather than depending on the discontinued public cloud.

The design is not yet a fully validated irrigation product. The final hardware wiring, pump integration, soil-specific calibration, and firmware-to-server connection remain pending. For that reason, the paper does not claim measured water savings, certified volumetric moisture accuracy, or successful remote pump control. These limits are important because a research report should separate a design intention from an experimentally observed result.

The next phase is to complete the low-voltage circuit, verify the pump and relay ratings, test the local server connection, collect repeatable sensor measurements in the actual soil, and record pump operation and delivered water volume. Once

those tests are complete, the prototype can be assessed quantitatively and then extended toward multi-zone irrigation using suitable low-power communication technologies.

B. References

[1] S. Datta and S. Taghvaeian, "Soil water sensors for irrigation scheduling in the United States: A systematic review of literature," Agricultural Water Management, vol. 278, art. 108148, 2023. doi: 10.1016/j.agwat.2023.108148.

[2] T. Kanso, M.-C. Gromaire, D. Ramier, P. Dubois, and G. Chebbo, "An Investigation of the Accuracy of EC5 and 5TE Capacitance Sensors for Soil Moisture Monitoring in Urban Soils—Laboratory and Field Calibration," Sensors, vol. 20, no. 22, art. 6510, 2020. doi: 10.3390/s20226510.

[3] A. A. Abd Halim et al., "IoT based smart irrigation, control, and monitoring system for chilli plants using NodeMCU-ESP8266," Bulletin of Electrical Engineering and Informatics, vol. 12, no. 5, 2023. doi: 10.11591/eei.v12i5.5266.

[4] F. R. Saputri et al., "Design and development of an irrigation monitoring and control system based on Blynk Internet of Things and ThingSpeak," PLOS ONE, vol. 20, no. 4, e0321250, 2025. doi: 10.1371/journal.pone.0321250.

[5] G. P. Pereira, M. Z. Chaari, and F. Daroge, "IoT-Enabled Smart Drip Irrigation System Using ESP32," IoT, vol. 4, no. 3, pp. 221-243, 2023. doi: 10.3390/iot4030012.

[6] Blynk Inc., "Migrating to the new Blynk: Full Guide," Blynk Documentation. Available: https://docs.blynk.io/en/troubleshooting/blynk-1.0-and-2.0-comparison/migrate-from-1.0-to-2.0. Accessed: 11 Nov. 2026.

[7] Peterkn2001, "blynk-server: fork of the discontinued Blynk Legacy Local Server," GitHub repository. Available: https://github.com/Peterkn2001/blynk-server. Accessed: 11 Nov. 2026.

[8] Ministry of Jal Shakti, Government of India, "Average Groundwater Consumption and Its Quality in the Country," Press Information Bureau, 10 Aug. 2026. Dynamic Ground Water Resources Assessment, 2025.

[9] Espressif Systems, "ESP8266EX Datasheet," version 7.1, Nov. 2025. Available: https://documentation.espressif.com/0a-esp8266ex_datasheet_en.html.

[10] Espressif Systems, "Introduction to ESP-NOW Scheme," ESP-Techpedia. Available: https://docs.espressif.com/projects/esp-techpedia/en/latest/esp-friends/solution-introduction/esp-now/esp-now-solution.html. Accessed: 11 Nov. 2026.

[11] LoRa Alliance, "What is LoRaWAN?" Available: https://lora-alliance.org/resource_hub/what-is-lorawan/. Accessed: 11 Nov. 2026.

C. Author contributions and acknowledgement

The work was carried out in the Department of Mechanical Engineering at Motilal Nehru National Institute of Technology Allahabad under the guidance of the IoT project instructor. The authors acknowledge the cited researchers, platform documentation, and open-source maintainers whose work supported the project.

D. Data and reproducibility statement

No controlled experimental dataset is claimed in this report. Raw readings, calibration statistics, pump run-time, and measured water volumes should be added after the remaining tests. The project record is available at https://iotproject-nit.vercel.app.

End of paper.